CC7711

Inteligência Artificial e Robótica

Prof. Dr. Flavio Tonidandel

Aprendizado de Máquina

Métodos Estatísticos de Classificação – parte 2

CC7711 - Inteligência Artificial e Robótica

SVM – Support Vector Machine

Máquina de vetor de suporte

SVM – Support Vector Machine

- Aprendizagem **Supervisionada**

- De forma geral, é usado para separação de classes **linearmente separáveis** *(classes que podem ser separadas por uma linha reta)*

  - A ideia é simples: **criar uma reta ou hiperplano, com margem máxima, que separa os dados em classes**

Support Vector Machine

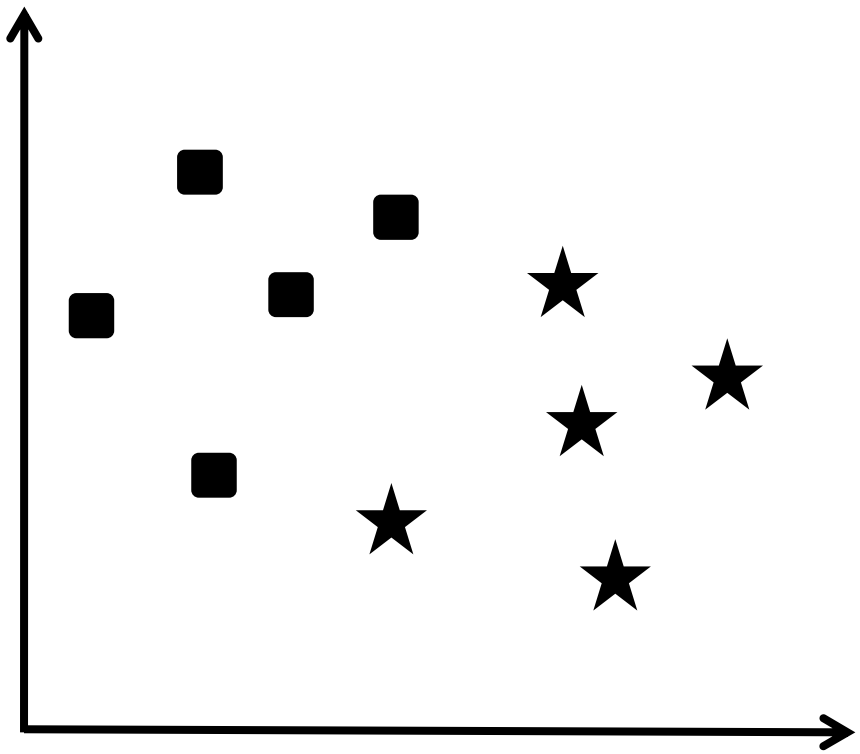
- Consiste em um método de aprendizado que tenta encontrar a **maior margem** para separar diferentes classes de dados.

- Pertence à classe de algoritmos de **aprendizado supervisionado**.

- A essência do SVM é a construção de um **hiperplano ótimo**, de modo que ele possa separar diferentes classes de dados com a maior margem possível.

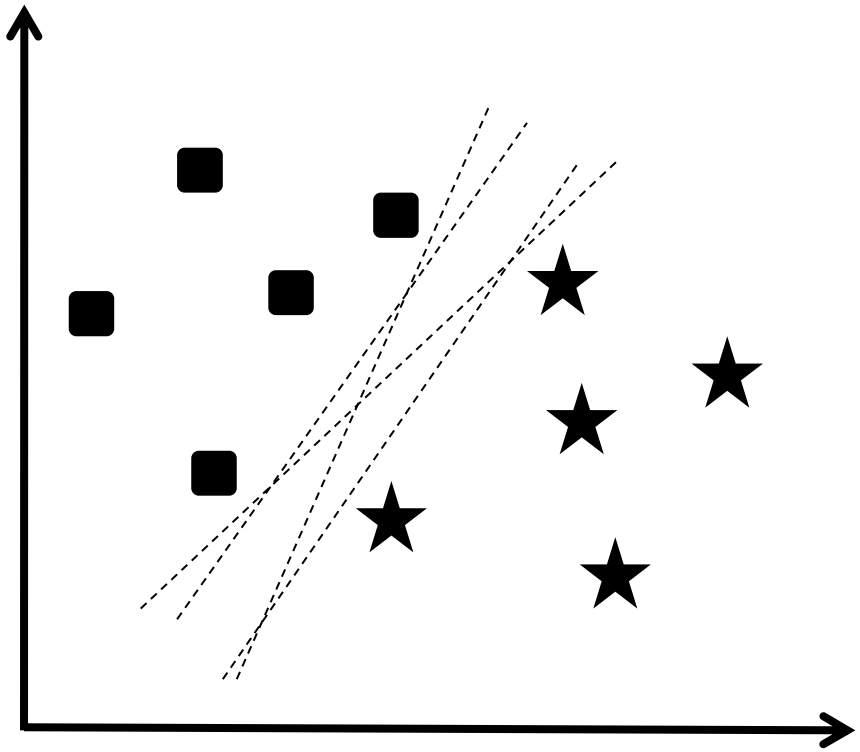
Support Vector Machine

- Como separar essas duas classes?



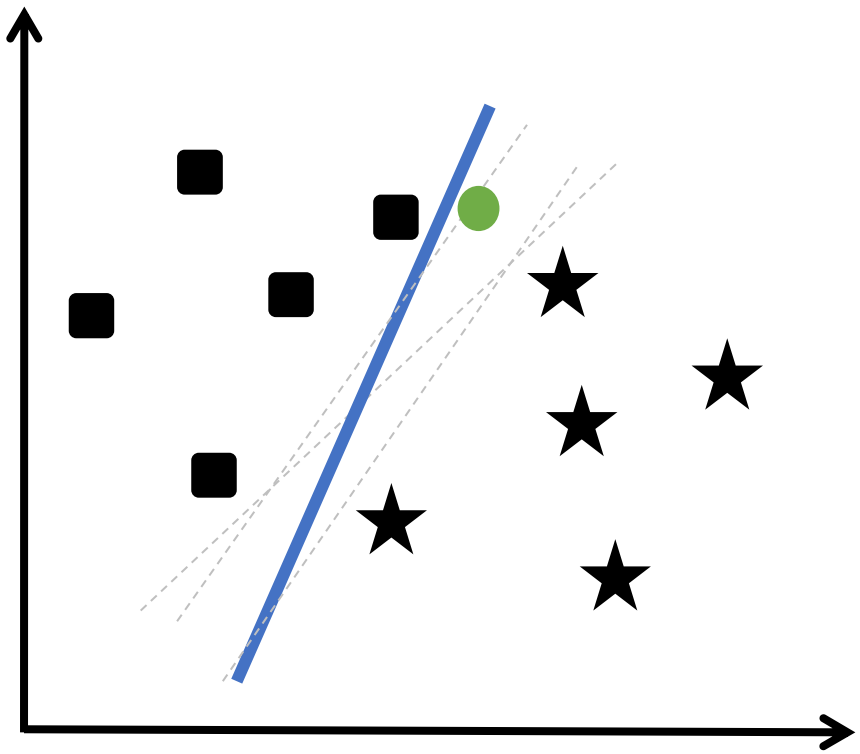
Support Vector Machine

- Como separar essas duas classes?
  - Existem diversas retas que podem ser traçadas para separar os dados.
- Qual delas é a melhor opção?



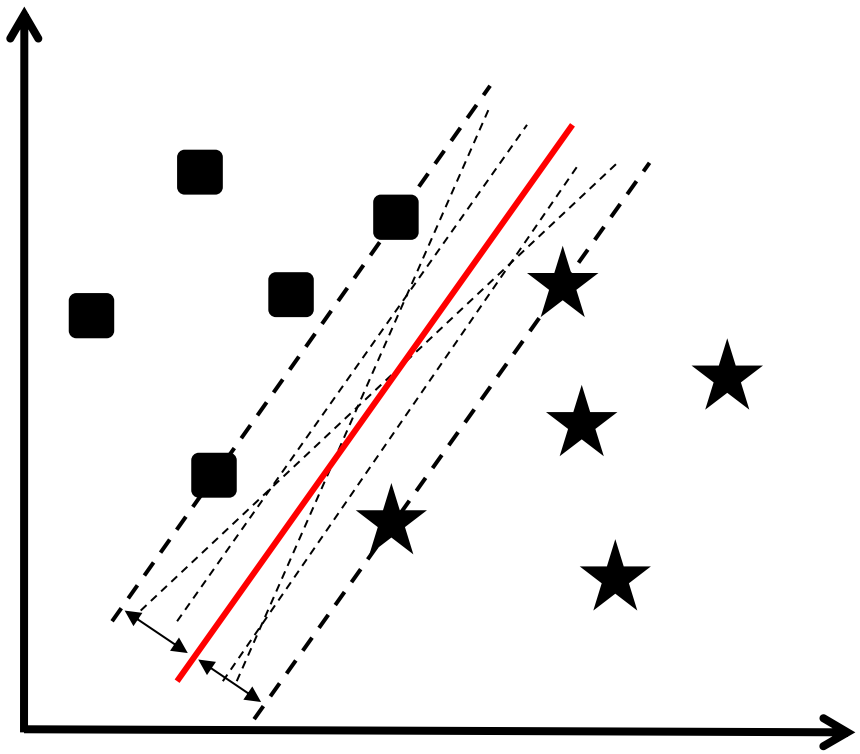
Support Vector Machine

- Como separar essas duas classes?
  - Existem diversas retas que podem ser traçadas para separar os dados.
- Qual delas é a melhor opção?
- Se considerarmos a reta **azul**, como classificaríamos o novo elemento **verde** ?
- a classificação do novo dado (**verde**) seria como **ESTRELA**... RUIM!



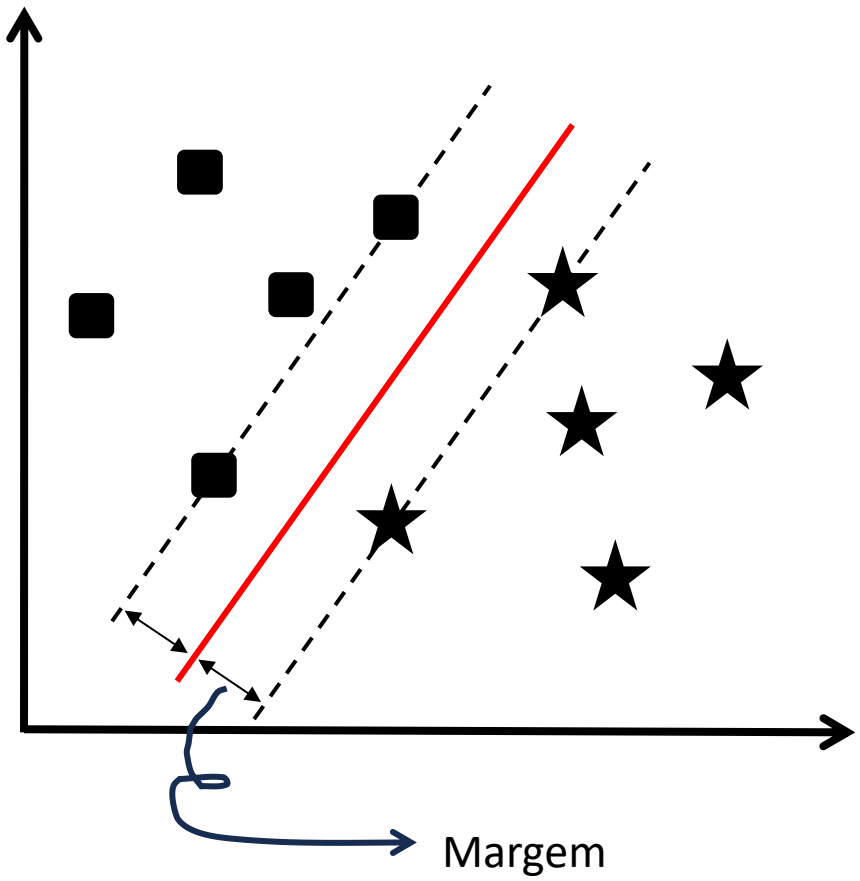
Support Vector Machine

- Como separar essas duas classes?
  - Existem diversas retas que podem ser traçadas para separar os dados.
- Qual delas é a melhor opção?
  - **Hiperplano ótimo!**



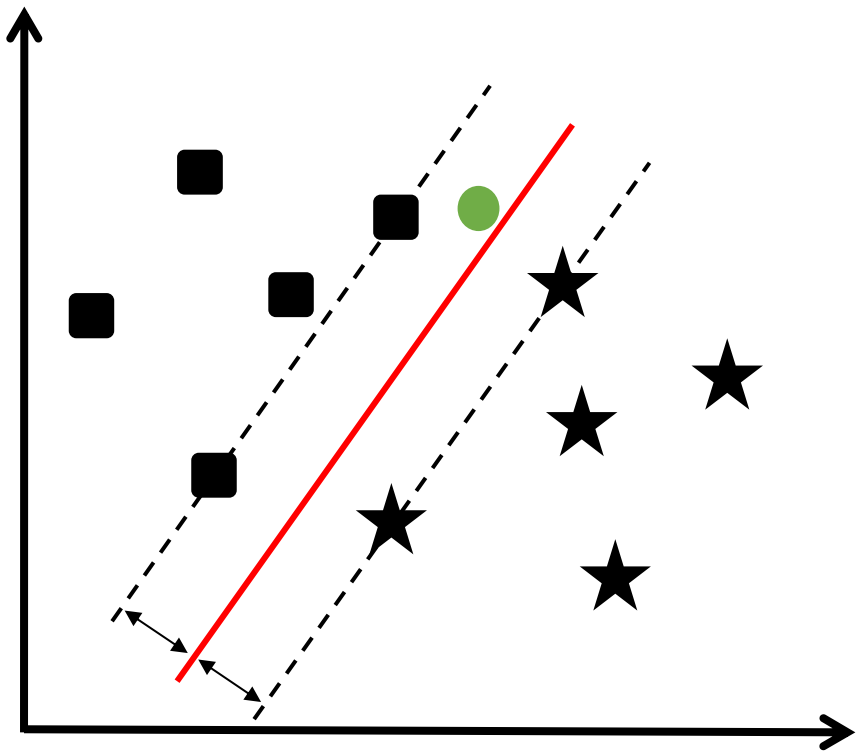
Support Vector Machine

- Como separar essas duas classes?
  - Existem diversas retas que podem ser traçadas para separar os dados.
- Qual delas é a melhor opção?
  - **Hiperplano ótimo!**
- A reta **vermelha** é melhor pois é a que está mais distante dos dois conjuntos.
- Isso faz com que o modelo tenha uma maior capacidade de **generalização**!



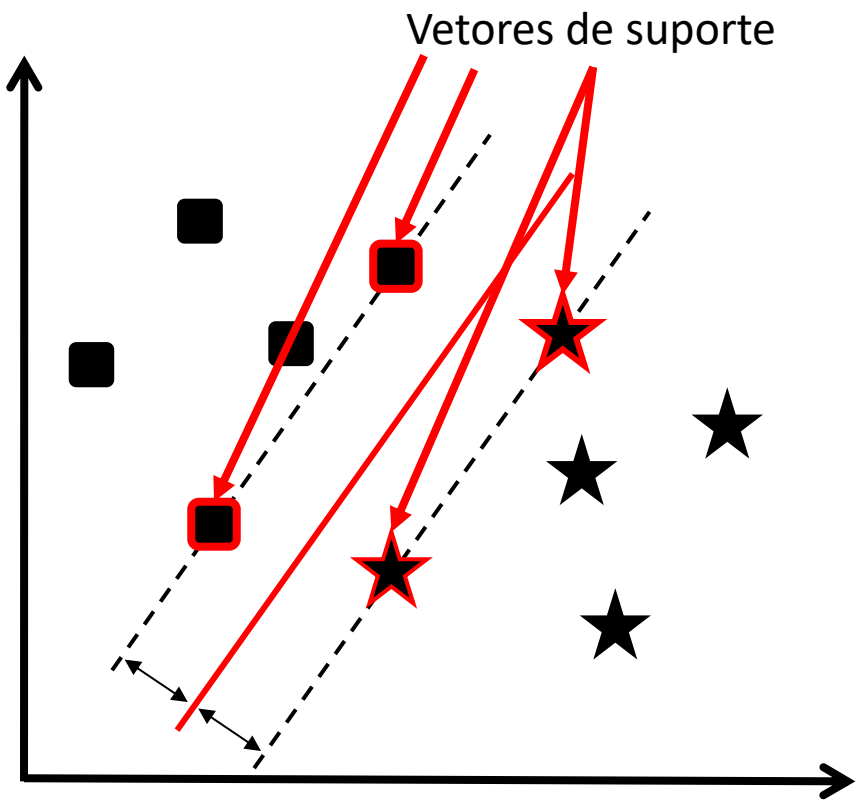
Support Vector Machine

- Como separar essas duas classes?
  - Existem diversas retas que podem ser traçadas para separar os dados.
- Qual delas é a melhor opção?
  - **Hiperplano ótimo!**
- O novo ponto **verde** se tornaria agora parte da classe dos quadrados!



Vetores de Suporte

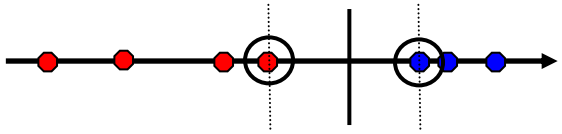
- Servem para definir qual será o hiperplano.
- São encontrados durante a fase de treinamento.
- Os vetores de suporte são os exemplos de treinamento realmente importantes. Os outros exemplos podem ser ignorados.



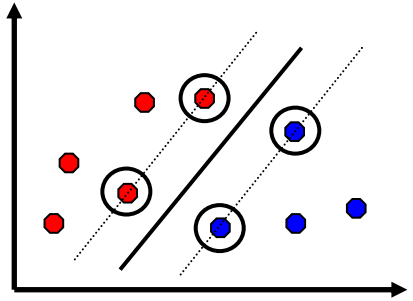
Support Vector Machine

Hiperplano:

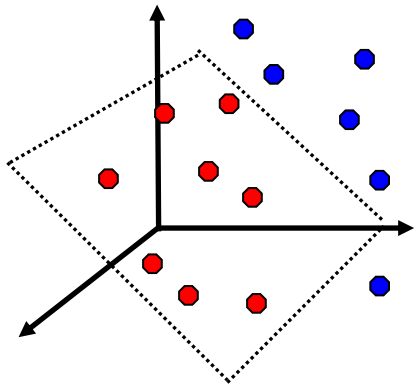
Espaço 1D = Ponto



Espaço 2D = Reta



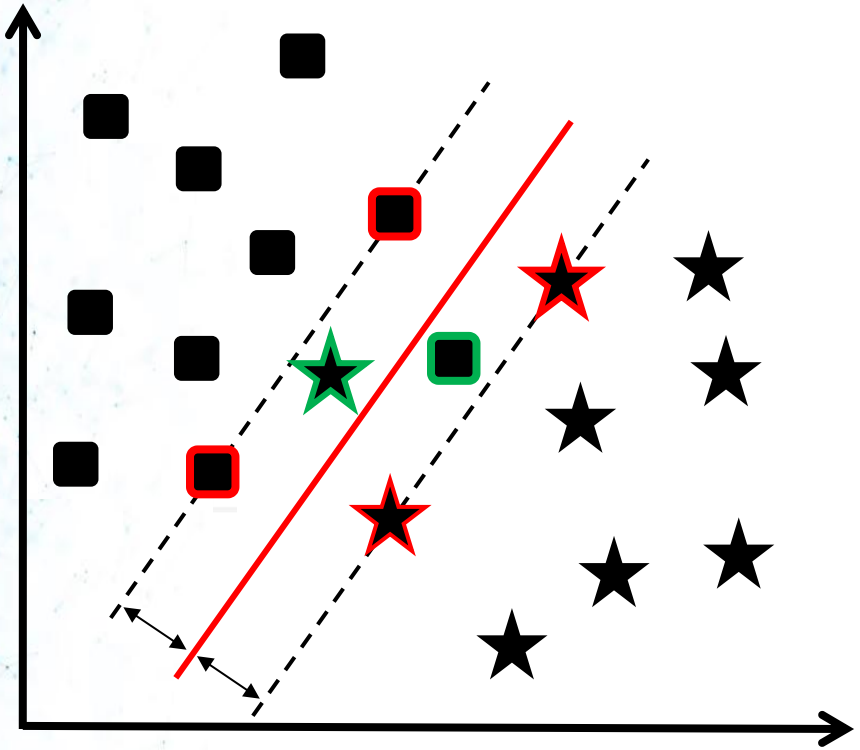
Espaço 3D = Plano



Support Vector Machine

- A aplicação de um método puramente linear para classificar um conjunto de dados pode sofrer com **dois problemas** bastante comuns:
  - **Outliers**
  - **Exemplos rotulados erroneamente**
- Mesmo assim o SVM ainda assim pode ser aplicado através do uso do parâmetro C (**soft margin** - variáveis de folga)

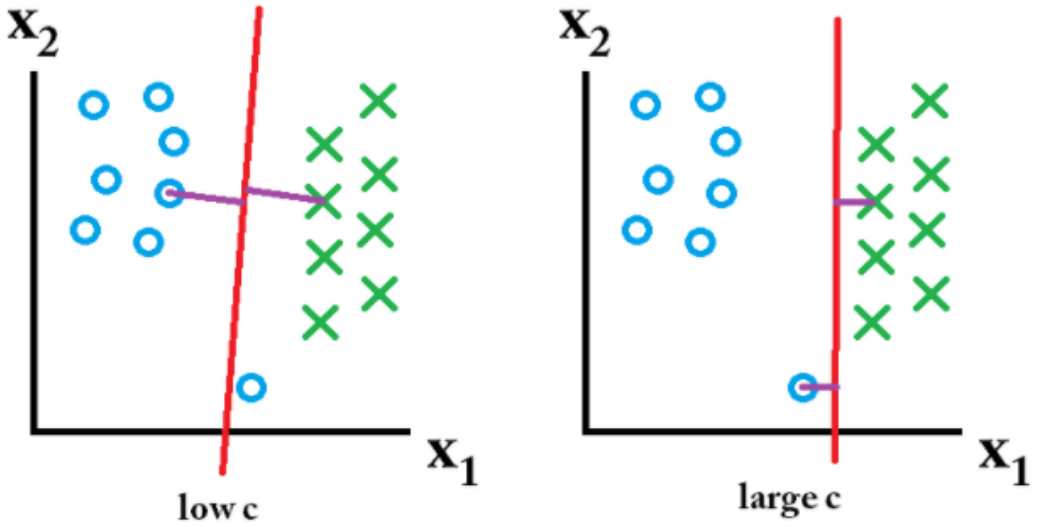
Soft Margin



Podemos considerar um custo da classificação incorreta **C**

Com este parâmetro, o SVM pode ignorar os outliers (ou exemplos rotulados indevidamente) na definição das margens

um valor baixo de C indica menos precisão, porém permite uma margem maior e vice-versa



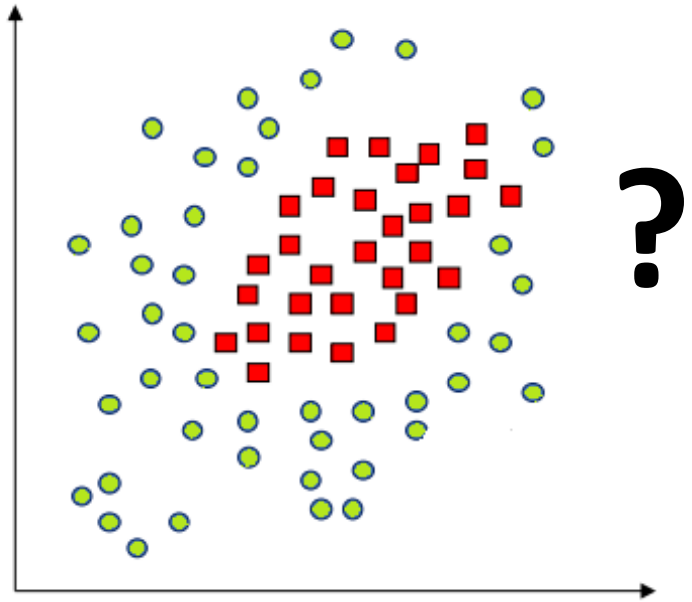
Support Vector Machine

- Em alguns problemas não é possível separar as classes linearmente mesmo utilizando a margem de folga.

- Na realidade, a grande maioria dos problemas reais não são separáveis linearmente.

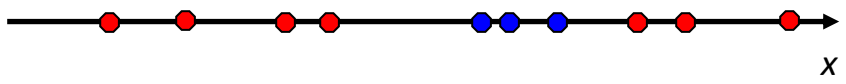
- O que fazer?

KERNEL TRICK



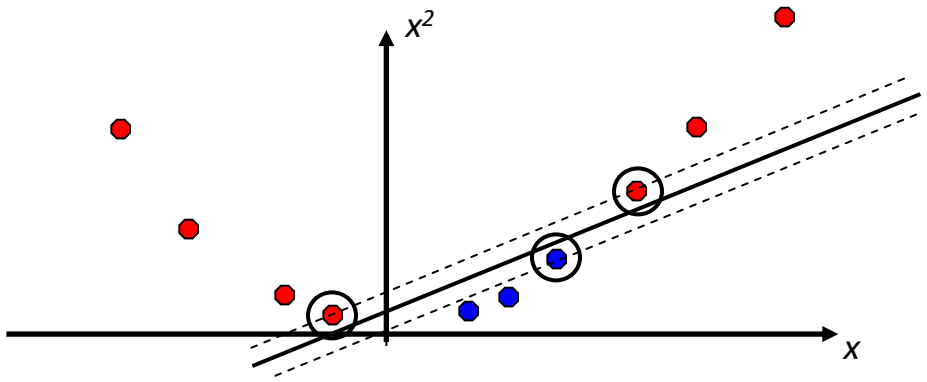
SVM Não-Linear

- O que fazer quando os dados não são linearmente separáveis?



1D

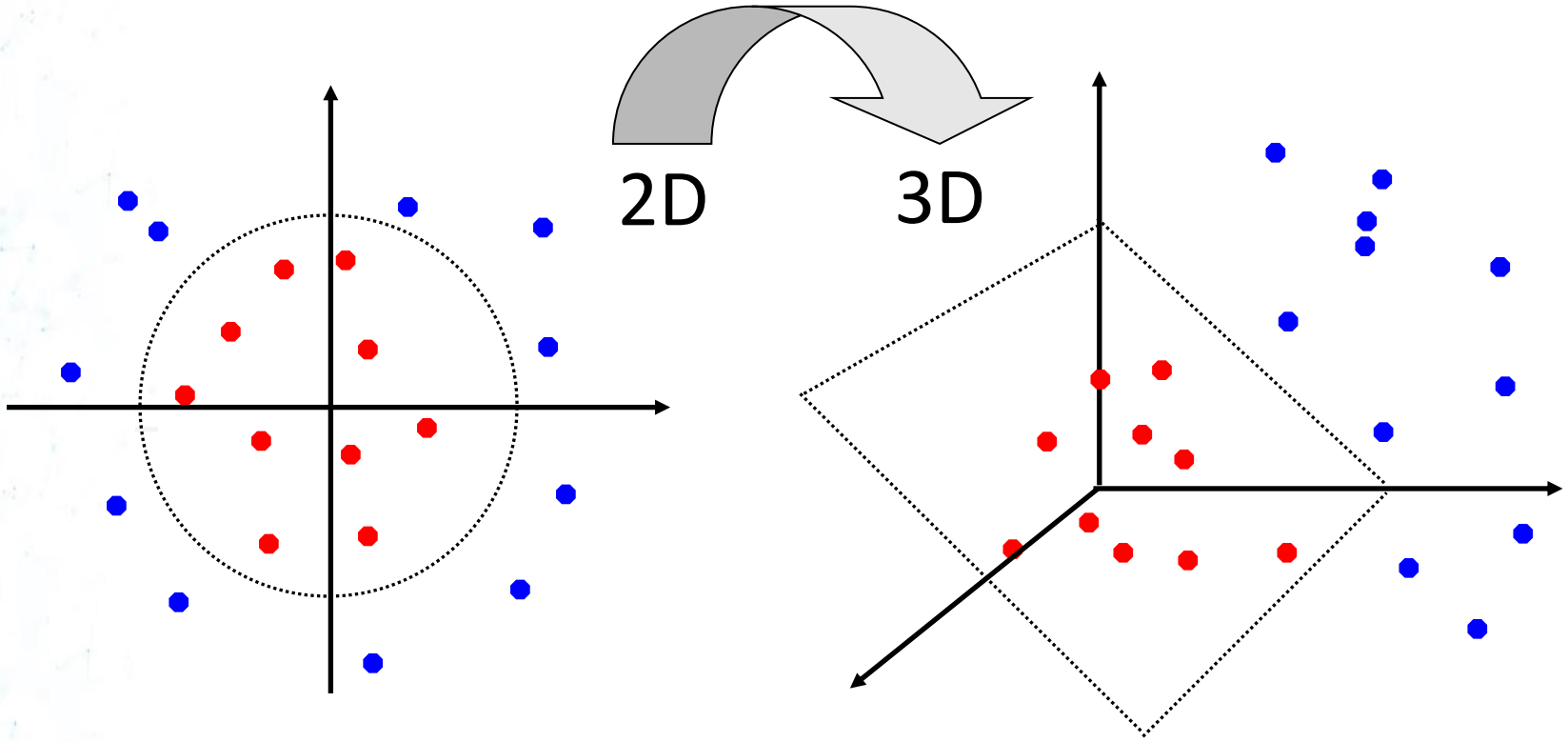
- A abordagem utilizada pelo SVM para resolver esse tipo de problema consistem em mapear os dados para um espaço de dimensão maior:



2D

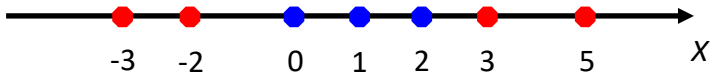
SVM Não-Linear

O espaço de atributos original pode ser mapeado em um espaço de atributos de dimensão maior onde o conjunto de treinamento se torna linearmente separável:

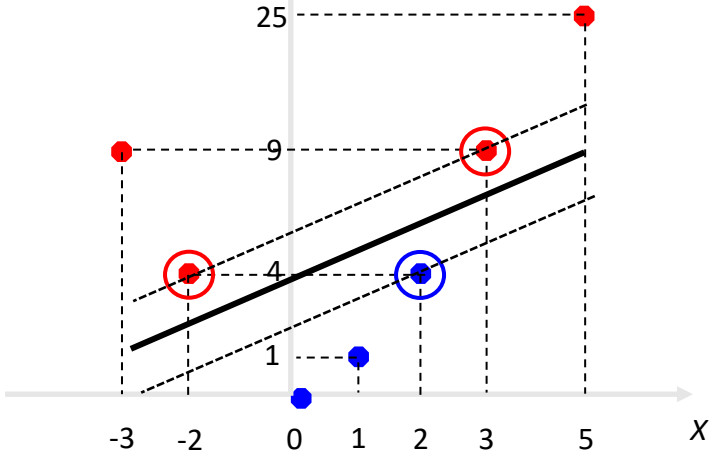


SVM Não-Linear Exemplo

- Considerando o seguinte conjunto de exemplos de treinamento que não são linearmente separáveis:



- Elevando para uma dimensão linearmente separável ($R^1 \rightarrow R^2$):



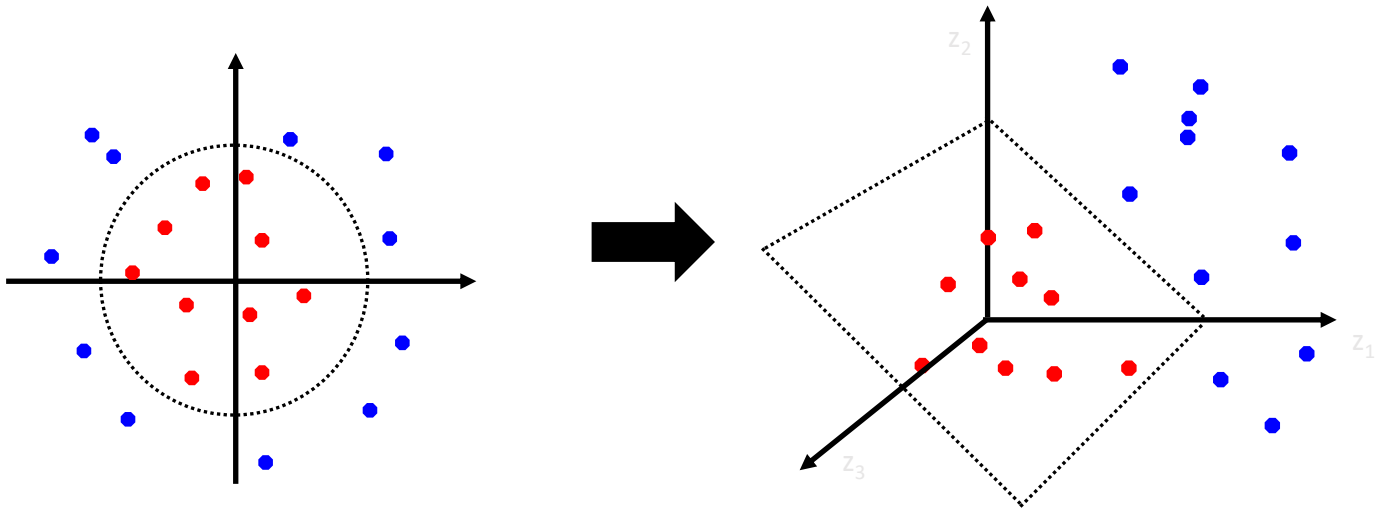
- **Kernel:** $\phi(x) = (x, x^2)$

SVM Não-Linear Exemplo

- A mesma metodologia pode ser aplicada em um espaço 2D de características ($R^2 \rightarrow R^3$), ou seja, de (x,y) para (x,y,z). Temos que ter uma função que mapeie x,y para x,y,z em termos de x,y

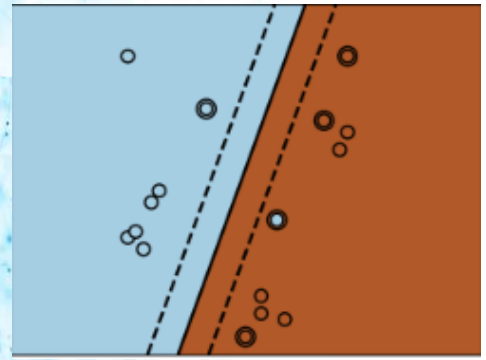
- A única diferença é a necessidade de uma nova função de kernel. Um exemplo de função de kernel aplicável nesse caso seria:

$$\phi(x_1, x_2) \rightarrow (z_1, z_2, z_3) = (x_1^2, \sqrt{2x_1x_2}, x_2^2)$$

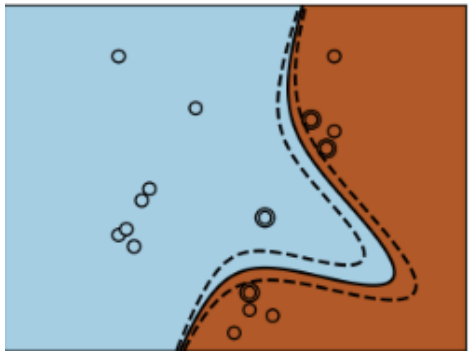


Tipos de Funções Kernel

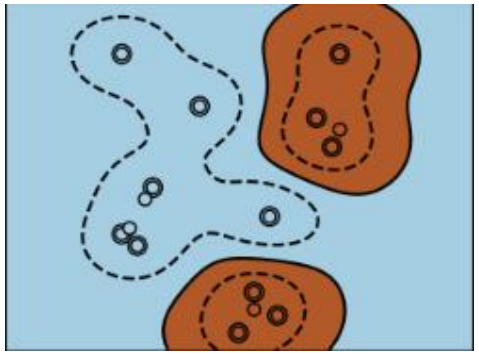
LINEARES

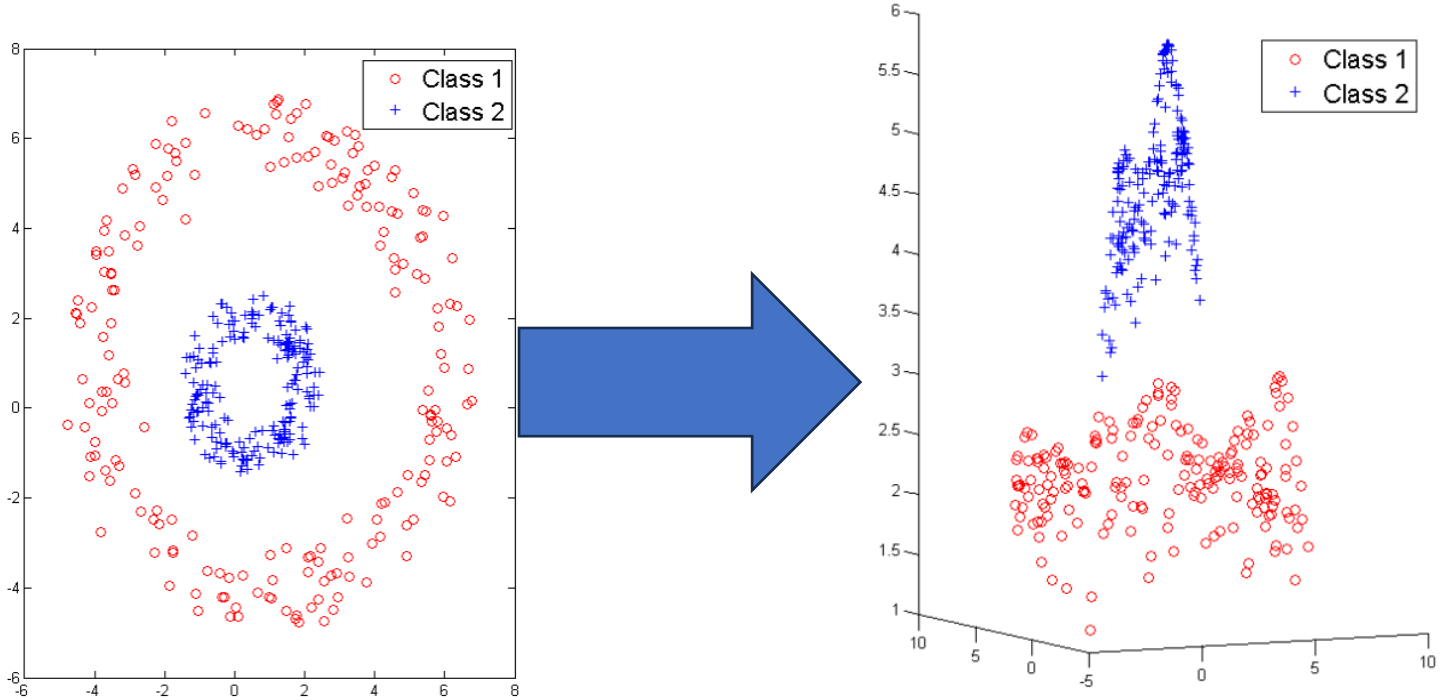


POLINOMIAIS



SIGMÓIDES





Fonte: https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html#sphx-glr-auto-examples-svm-plot-svm-kernels-py

Support Vector Machine

- **Vantagens:**
  - Consegue lidar bem com grandes conjuntos de exemplos.
  - Trata bem dados de alta dimensão.
  - O processo de classificação é rápido.

- **Desvantagens:**
  - É necessário definir um bom Kernel.
  - O tempo de treinamento pode ser bem longo dependendo do número de exemplos e dimensionalidade dos dados.

Support Vector Machine

- **Como o SVM encontra as margens ótimas considerando o parâmetro de soft margin C ?**
- Este cálculo é bem complexo e foge do escopo desta aula. Existem funções prontas que já realizam estes cálculos e que podemos usar:

Utilizando a Biblioteca *scikit-learn*

Support Vector Machine

De *sklearn* vamos importar *svm*

```
from sklearn import svm
```

usar o *SVC - Support Vector Classification*

```
sv = svm.SVC(kernel='poly', degree=5, gamma='auto', C=C)
```

kernel: especifica o tipo de kernel (“poly” é o polinomial – mais comum)

degree: para kernel polinomial - grau

gamma: é o parâmetro de um Kernel Gaussiano - um alto valor de gama leva a mais precisão, porém cria ilhas de decisão

C: o custo da classificação incorreta - um valor baixo de C indica menos precisão, porém permite uma margem maior e vice-versa

Para a especificação de mais parâmetros, e exemplos de uso, consulte:
https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

Bagging

Bootstrap

A ideia básica:

Sorteie aleatoriamente conjuntos de dados, a partir dos dados originais, criando um conjunto de dados (amostra) que serão usados como dados de treinamento. Cada amostra tem o mesmo tamanho que o conjunto de treinamento original, i.e., pode conter dados repetidos.

Amostragem com substituição
Sampling with replacement
cada selecionado é "colocado" de volta no pote e pode ser selecionado de novo.

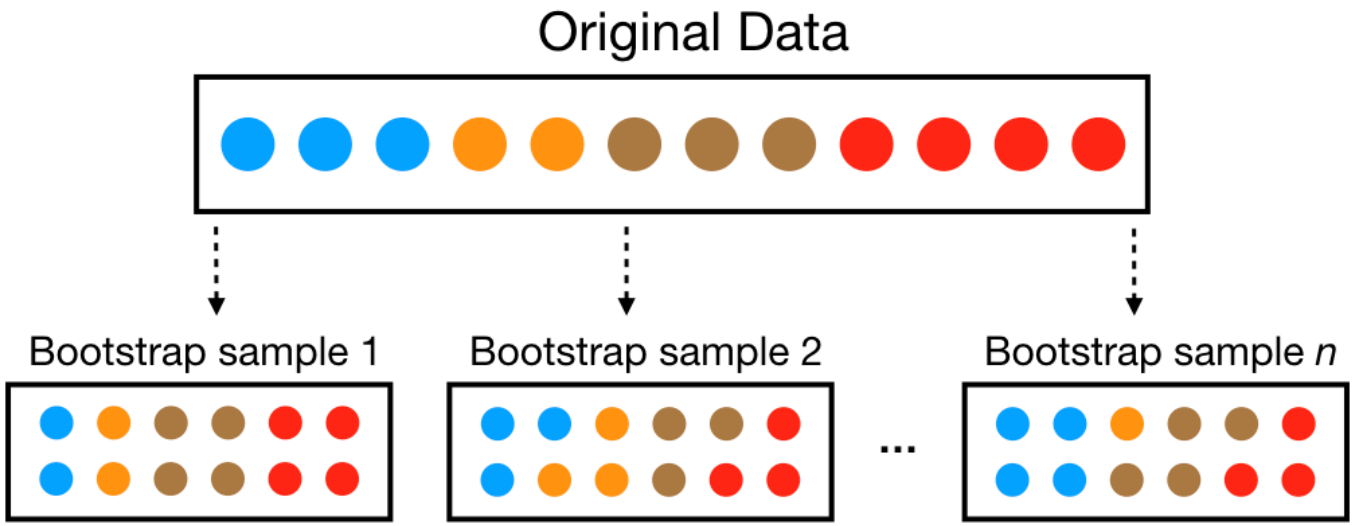


Image from datasciencedojo.com

Bagging

Dados Originais

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

N dados

Bootstrap Amostra 1

X	Y
X_4	y_4
X_{14}	y_{14}
X_{11}	y_{11}
X_2	y_2
X_{35}	y_{35}
$\vdots$	$\vdots$
X_k	y_k

N escolhidos
Pode ter repetido

Decision Tree 1

Dados usados

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Based on slides of CS109A Introduction to Data Science - IACS - Harvard University

Bagging

Dados Originais

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

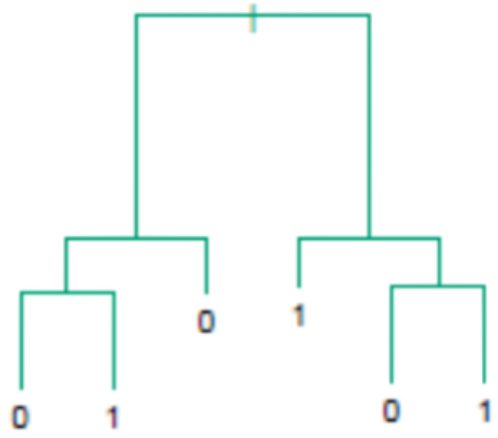
N dados

Bootstrap Amostra 2

X	Y
X_5	y_5
X_3	y_3
X_{12}	y_{12}
X_{43}	y_{43}
X_1	y_1
$\vdots$	$\vdots$
X_k	y_k

N escolhidos
Pode ter repetido

Decision Tree 2



Dados usados

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Based on slides of CS109A Introduction to Data Science - IACS - Harvard University

Bagging

Dados Originais

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

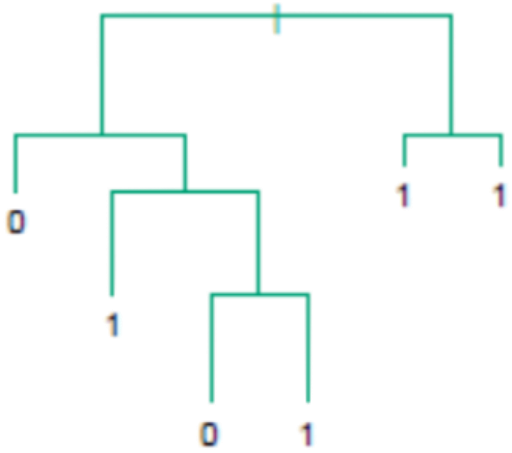
N dados

Bootstrap Amostra 3

X	Y
X_9	y_9
X_4	y_4
X_1	y_1
X_1	y_1
X_{65}	y_{65}
$\vdots$	$\vdots$
X_k	y_k

N escolhidos
Pode ter repetido

Decision Tree 3



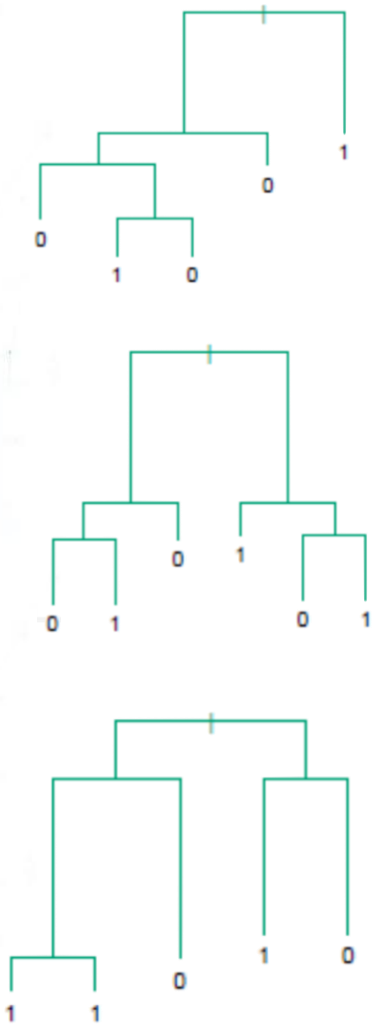
Dados usados

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
X_4	y_4
X_5	y_5
$\vdots$	$\vdots$
X_n	y_n

Based on slides of CS109A Introduction to Data Science - IACS - Harvard University

Classificação

X	Y
X_1	y_1
X_2	y_2
X_3	y_3
$\vdots$	$\vdots$
X_i	y_i
$\vdots$	$\vdots$
X_n	y_n

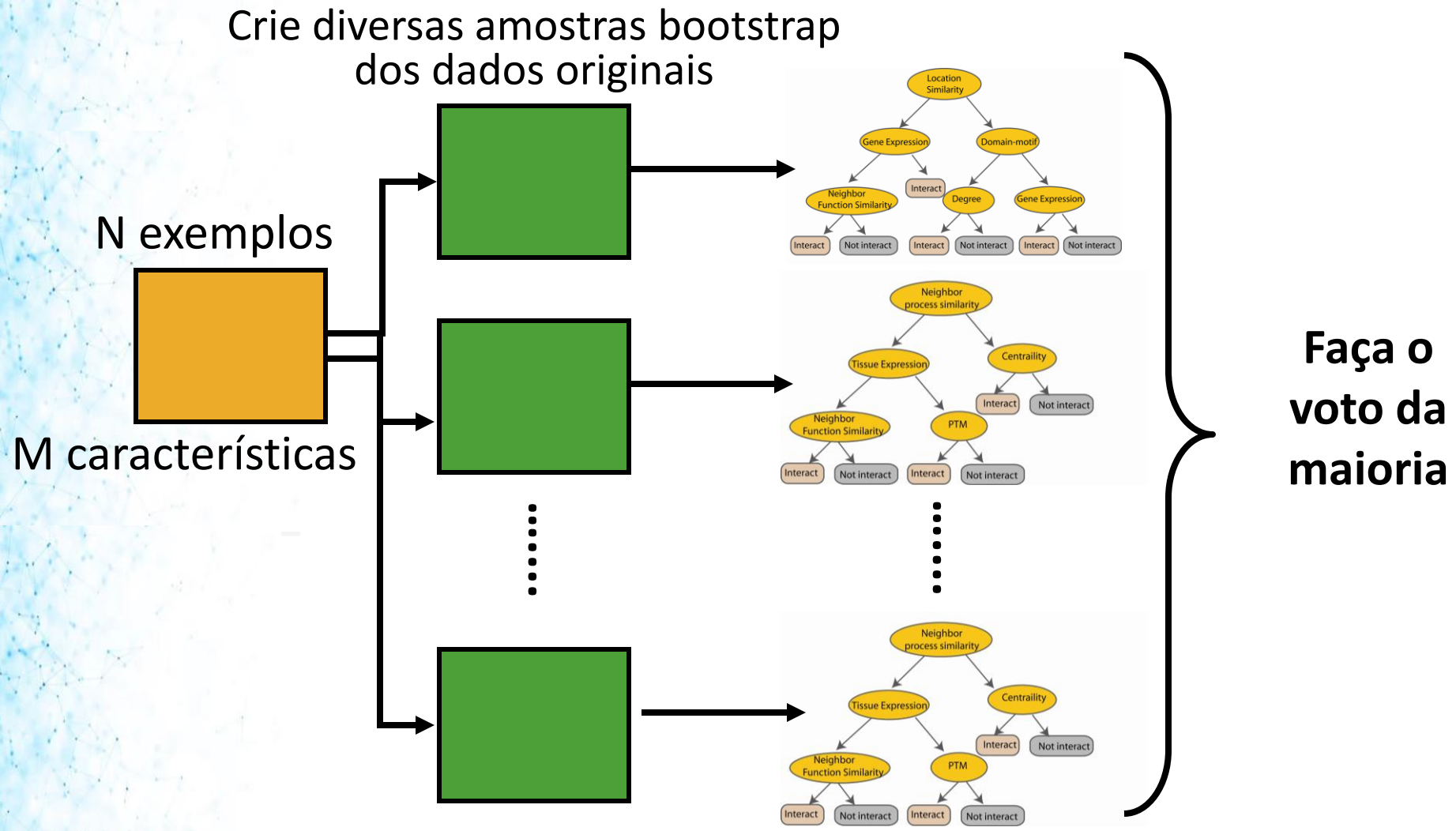


Classificação

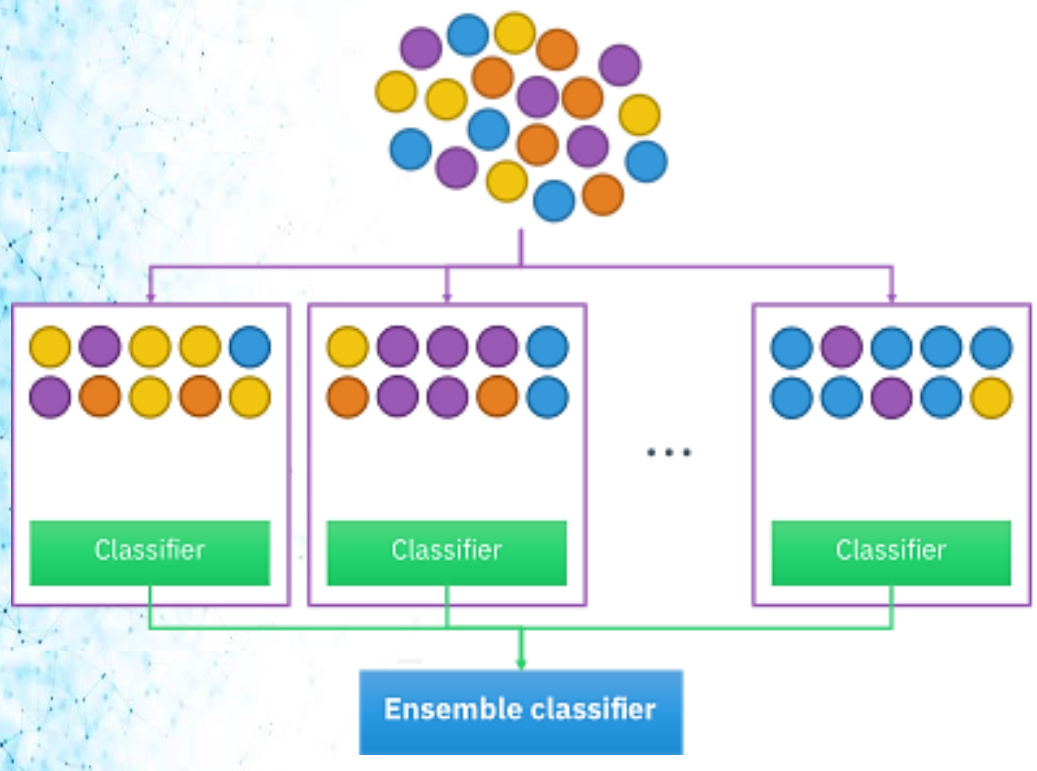
$$\hat{y}_{i,pw} = \text{Maioria}(\hat{y}_i)$$

Based on slides of CS109A Introduction to Data Science - IACS - Harvard University

Classificador Bagging



Bagging (método Geral)



Original Data

Bootstrapping

Classificadores (modelos)

Bagging

O ***Bagging*** é um exemplo de ***Ensemble Method (***método de conjunto), um método de construir um único modelo treinando e agregando vários modelos.

Bagging

- **Quais os problemas do Bagging** ?
- Pode ocorrer overfitting se as árvores são muito grandes.
- Se as árvores forem muito rasas, podem ser insuficientes. subajustar.

- Outro ponto importante:
  - Na prática, os conjuntos de árvores no Bagging tendem a ser altamente correlacionadas. Isto porque todas as árvores tenderão a escolher o atributo de maior ganho (ver aula de aprendizado indutivo) como o primeiro nó da árvore.
  - Ou seja, cada árvore do conjunto acaba sendo distribuída de forma idêntica, com a saída esperada do modelo médio igual à saída esperada de qualquer uma das árvores.

Bagging

- **Quais os problemas do Bagging** ?

- Pode ocorrer overfitting se as árvores são muito grandes.
- Se as árvores forem muito rasas, podem ser insuficientes. subajustar.

- Outro ponto importante:
  - Na prática, os conjuntos de árvores no Bagging tendem a ser altamente correlacionadas. Isto porque todas as árvores tenderão a escolher o atributo de maior ganho (ver aula de aprendizado indutivo) como o primeiro nó da árvore.
  - Ou seja, cada árvore do conjunto acaba sendo distribuída de forma idêntica, com a saída esperada do modelo médio igual à saída esperada de qualquer uma das árvores.

PODEMOS MELHORAR !!!!

Random Forest

Random Forest

***Random Forest** é uma forma modificada de **Bagging** que cria conjuntos de árvores de decisão independentes.*

Para descorrelacionar as árvores, deve-se:

- *treinar cada árvore em uma amostra de bootstrap separada do conjunto de treinamento completo **(o mesmo que no Bagging)***
- *para cada árvore, em cada amostra, selecionamos aleatoriamente um sub-conjunto de atributos do conjunto completo de atributos.*
- *Dentre os atributos do sub-conjunto, selecionamos o atributo de maior ganho correspondente para a amostra, e assim formar a arvore de decisão.*

Random Forest

Dados Originais

Ai são os atributos

A1	A2	A3	A4	A5	A6	A7	Classe
97	91	5	17	46	65	2	Yes
89	36	10	6	51	86	2	Yes
253	24	6	28	14	107	73	No
104	70	5	5	51	9	178	No
20	38	8	15	76	473	17	Yes
326	11	9	20	8	281	196	Yes
374	89	3	10	89	476	40	No
377	28	7	20	78	188	28	No
76	22	5	1	100	151	157	Yes
73	86	0	14	45	180	98	Yes
79	75	5	29	47	66	49	No

Os modelos de Random Forest têm vários parâmetros para ajustar, os mais comuns são:

- o número de atributos a serem selecionados aleatoriamente em cada bootstrap
  Geralmente usa-se **$\sqrt{N}$ (arredondado para baixo)** onde N é a quantidade de atributos

- o número total de árvores no conjunto

Random Forest

Dados Originais

Ai são os atributos

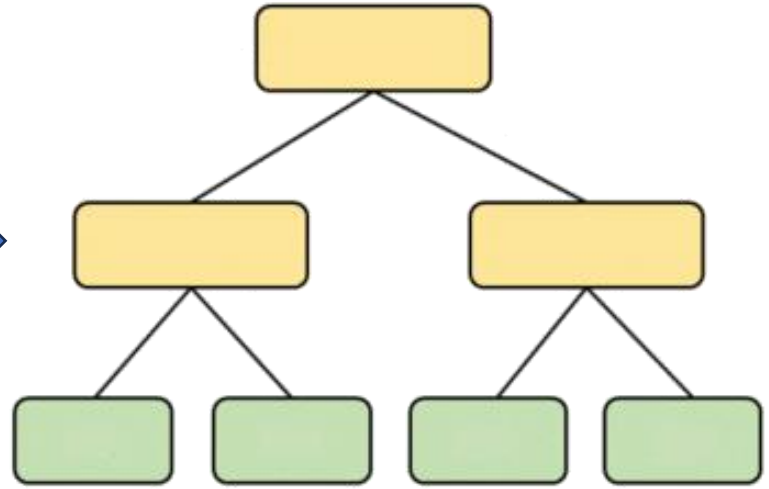
A1	A2	A3	A4	A5	A6	A7	Classe
97	91	5	17	46	65	2	Yes
89	36	10	6	51	86	2	Yes
253	24	6	28	14	107	73	No
104	70	5	5	51	9	178	No
20	38	8	15	76	473	17	Yes
326	11	9	20	8	281	196	Yes
374	89	3	10	89	476	40	No
377	28	7	20	78	188	28	No
76	22	5	1	100	151	157	Yes
73	86	0	14	45	180	98	Yes
79	75	5	29	47	66	49	No

Amostragem com substituição
Sampling with replacement

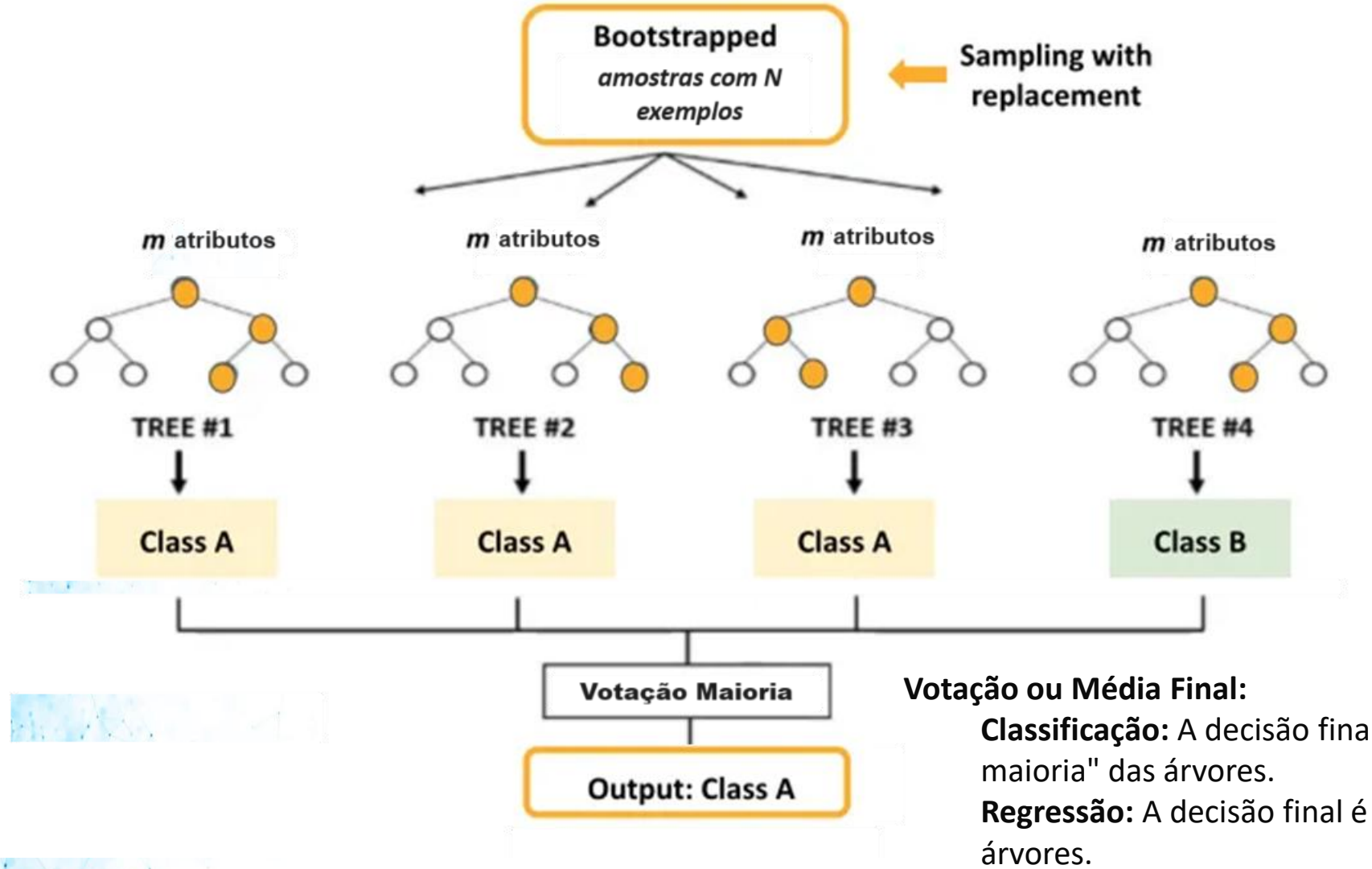
A2	A6	Classe
36	86	Yes
38	473	Yes
11	281	Yes
28	188	No
22	151	Yes
86	180	Yes
11	281	Yes
86	180	Yes
11	281	Yes
38	473	Yes
38	473	Yes

Random Forest

A2	A6	Classe
36	86	Yes
38	473	Yes
11	281	Yes
28	188	No
22	151	Yes
86	180	Yes
11	281	Yes
86	180	Yes
11	281	Yes
38	473	Yes
38	473	Yes



Random Forest



Votação ou Média Final:

Classificação: A decisão final é tomada pela "votação da maioria" das árvores.

Regressão: A decisão final é a média das previsões de todas as árvores.

Imagem adaptada de AIML.com Research

Random Forest

Cenário típico

- Boa generalização e interpretabilidade “prática”.

Quando Bagging funciona

Quando uma árvore única causa overfit.
O ensemble “faz média” das excentricidades.

Benefício do Random Forest

Redução do Overfitting: Como várias árvores são treinadas e seus resultados combinados, o risco do modelo memorizar o ruído dos dados de treinamento é drasticamente reduzido.

Alta Precisão e Estabilidade: A combinação de modelos fracos (individualmente) resulta em um modelo forte, estável e com maior poder de generalização para novos dados.

Exemplo de codigo (Python)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,
    max_features='sqrt',
    min_samples_leaf=10,
    class_weight='balanced',
    oob_score=True,
    n_jobs=-1,
    random_state=42
)
rf.fit(X_train, y_train)

print(rf.oob_score_)
proba = rf.predict_proba(X_test)[:, 1]
```

Análise do Random Forests

Quando o número de atributos for grande, mas o número de atributos relevantes for pequeno, o Random Forest pode ter um desempenho ruim.

Por quê?

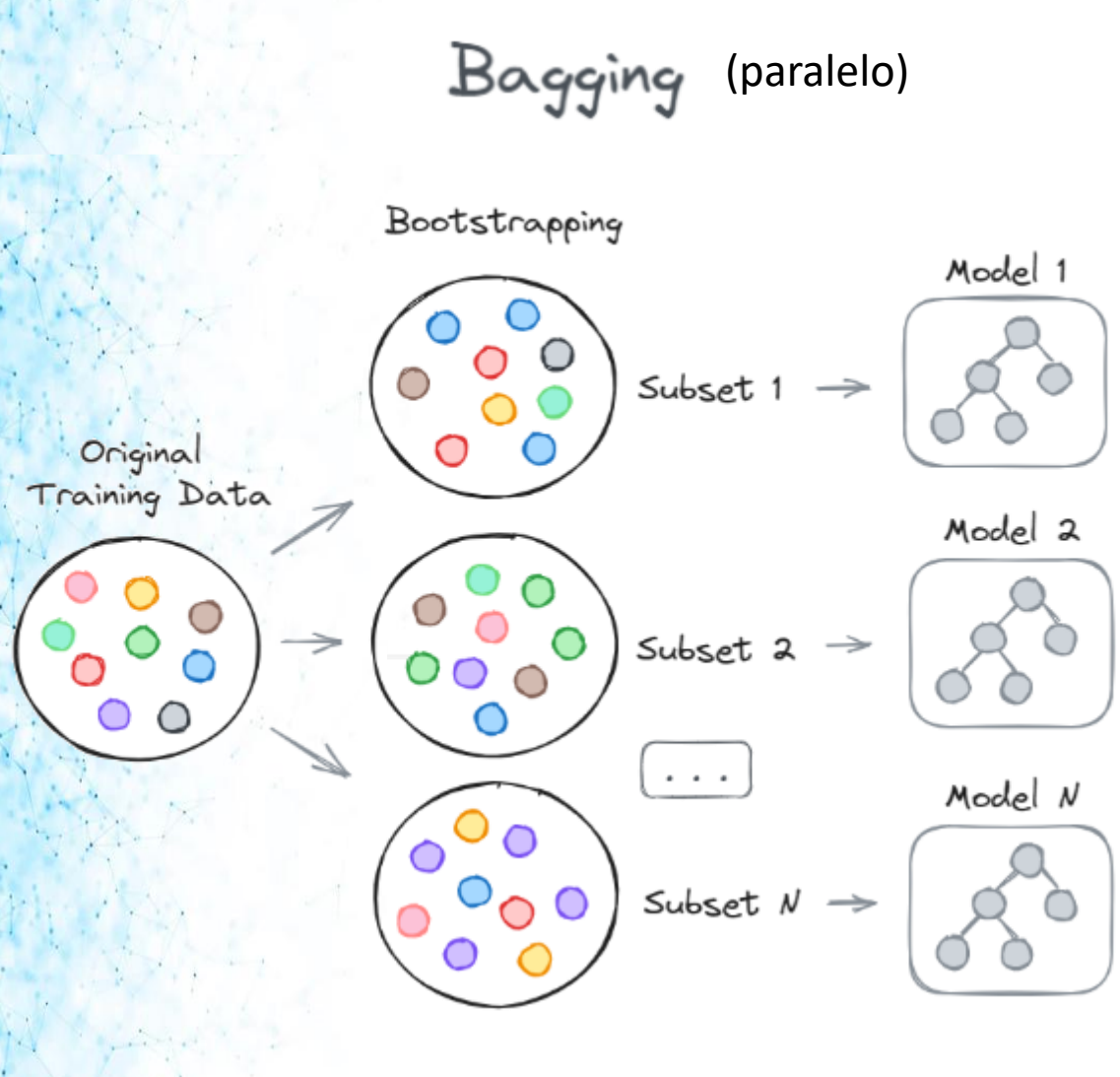
Em cada divisão, as chances de selecionar um atributo relevante serão baixas e, portanto, a maioria das árvores no conjunto serão modelos fracos.

Para estes casos existe alguma solução ?

Sim ! Existem outras técnicas que abordam bem número grande de atributos, porém com poucos relevantes → BOOSTING

Boosting

Bagging x Boosting



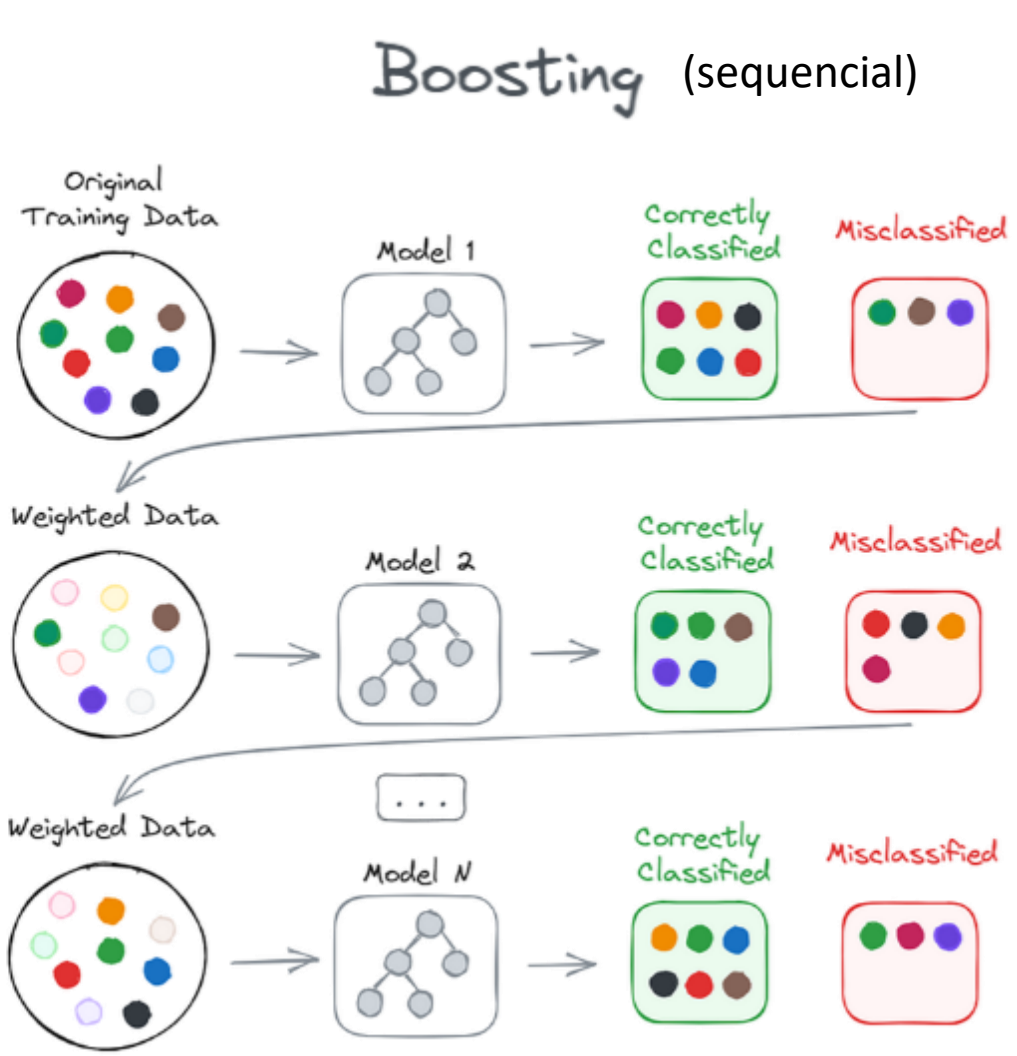


Imagem retirada de: https://medium.com/@brijesh_soni/understanding-boosting-in-machine-learning-a-comprehensive-guide-bdeaa1167a6

Boosting (processo)

Enquanto no Bagging resolve-se o problema com “muitos em paralelo”, no Boosting o conceito é “um aprendendo com o outro”

Inicializar pesos: *No início do processo, cada exemplo de treinamento recebe o mesmo peso.*

By Sirakorn - Wikimedia

Boosting (processo)

Enquanto no Bagging resolve-se o problema com “muitos em paralelo”, no Boosting o conceito é “um aprendendo com o outro”

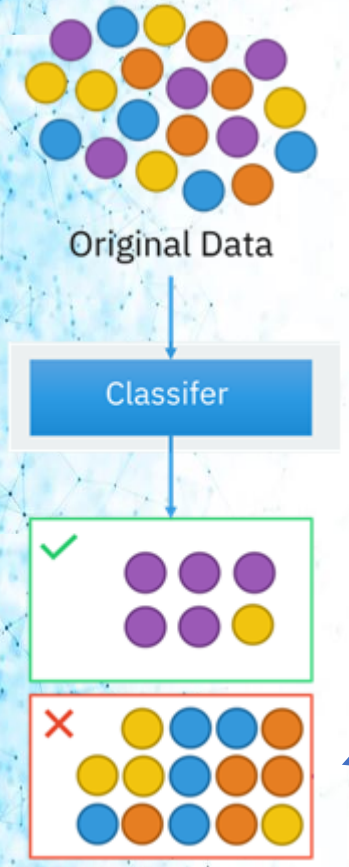
Inicializar pesos

Treinar um aprendizado fraco*: Os dados de treinamento ponderados são usados para treinar um aprendizado fraco, que é um modelo simples que supera a adivinhação aleatória apenas marginalmente. Uma árvore de decisão com alguns níveis, por exemplo, pode ser empregada como um aprendizado fraco.*

By Sirakorn - Wikimedia

Boosting (processo)

Enquanto no Bagging resolve-se o problema com "muitos em paralelo", no Boosting o conceito é "um aprendendo com o outro"



Inicializar pesos

Treinar um aprendizado fraco

Cálculo de erro: *o erro do aprendizado fraco nos dados de treinamento é computado. A soma ponderada dos casos mal classificados constitui o erro.*

By Sirakorn - Wikimedia

Boosting (processo)

Enquanto no Bagging resolve-se o problema com "muitos em paralelo", no Boosting o conceito é "um aprendendo com o outro"

Inicializar pesos

Treinar um aprendizado fraco

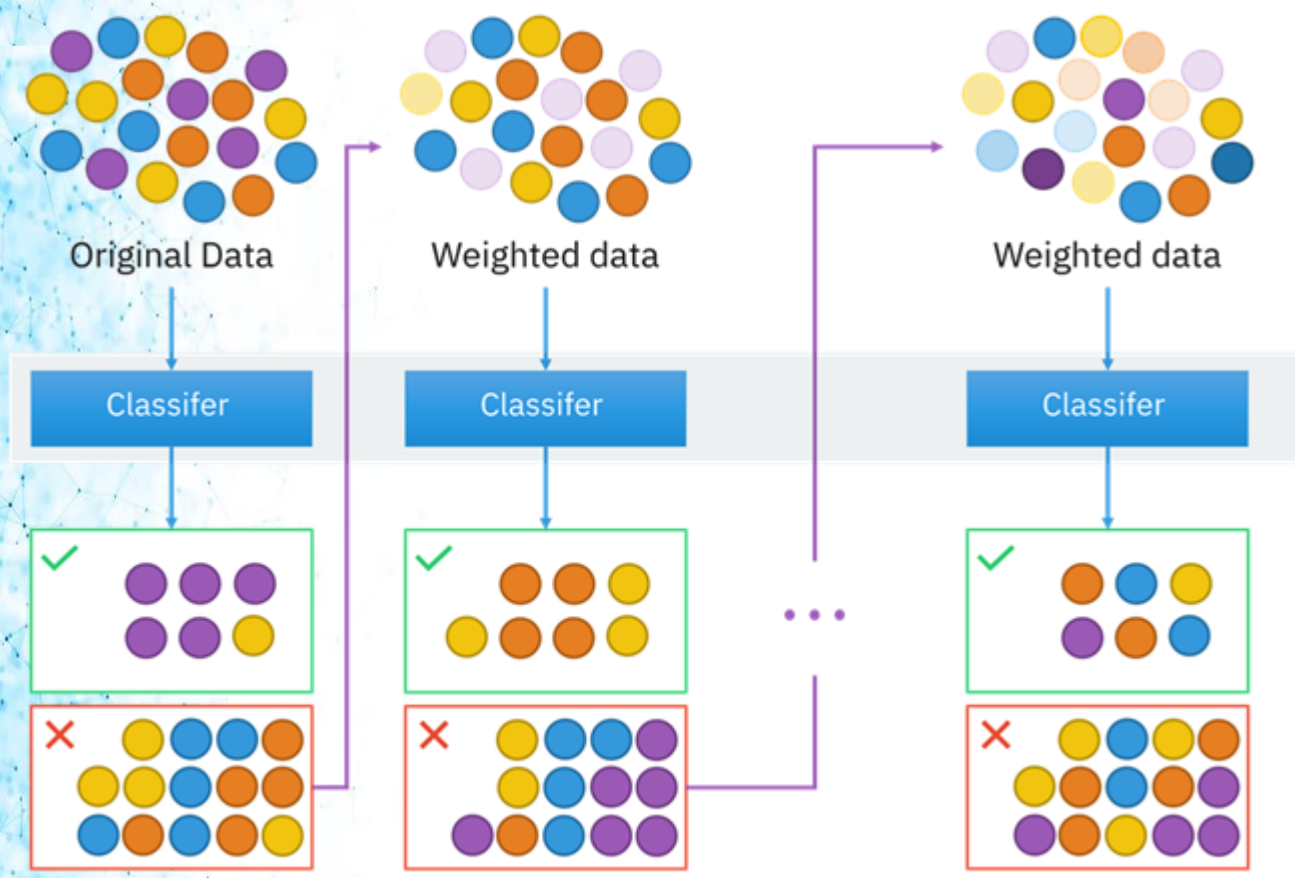
Cálculo de erro

Atualizar pesos*: Os pesos são atualizados de acordo com a taxa de erro dos exemplos de treinamento. Exemplos mal classificados recebem pesos mais altos, enquanto exemplos corretamente classificados recebem pesos mais baixos.*

By Sirakorn - Wikimedia

Boosting (processo)

Enquanto no Bagging resolve-se o problema com "muitos em paralelo", no Boosting o conceito é "um aprendendo com o outro"



Inicializar pesos

Treinar um aprendizado fraco

Cálculo de erro

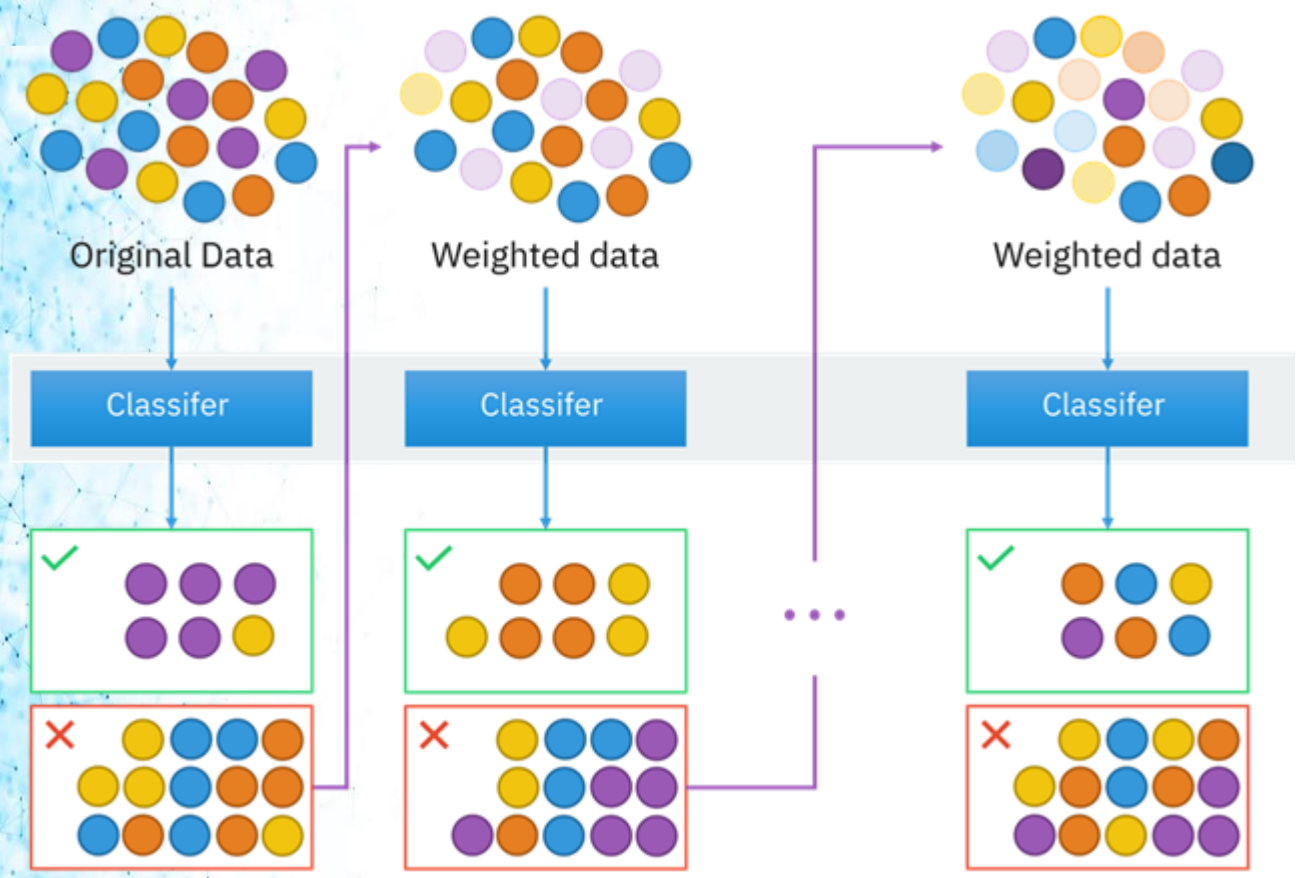
Atualizar pesos

Repetir: *as etapas de 2 a 4 são repetidas várias vezes. Um novo aprendizado fraco é treinado sobre os pesos atualizados dos exemplos de treinamento em cada ciclo.*

By Sirakorn - Wikimedia

Boosting (processo)

Enquanto no Bagging resolve-se o problema com "muitos em paralelo", no Boosting o conceito é "um aprendendo com o outro"



Inicializar pesos

Treinar um aprendizado fraco

Cálculo de erro

Atualizar pesos

Repetir

Combine aprendizados fracos: *O modelo final é composto por todos os aprendizados fracos que foram treinados nas etapas anteriores. A precisão de cada aprendizado fraco é ponderada, e a previsão final é baseada no total ponderado de cada aprendizado fraco.*

By Sirakorn - Wikimedia

AdaBoost

AdaBoost (Adaptive Boosting): AdaBoost é um dos algoritmos de Boosting mais amplamente usados. Ele fornece pesos para cada ponto de dados no conjunto de treinamento com base na precisão dos modelos anteriores e, em seguida, treina um novo modelo usando os pesos atualizados.

StatQuest Youtube Page: AdaBoost, Clearly Explained !!
https://www.youtube.com/watch?v=LsK-xG1cLYA

O modelo continua otimizado de forma sequencial até produzir o preditor mais forte.

Se o processo é contínuo.... COMO OU QUANDO PARAR ???

By Sirakorn - Wikimedia

AdaBoost (critério de Parada)

Número Máximo de Iterações (n_estimators): O treinamento para quando um número predefinido de classificadores fracos é alcançado.

Erro de Treinamento Zero (ou muito baixo): Se o conjunto de classificadores fracos conseguir classificar perfeitamente todos os dados de treinamento, o algoritmo pode parar antes de atingir o número máximo de iterações, pois não há mais erros para corrigir.

Melhoria Mínima do Erro: O processo iterativo continua até que nenhuma melhora adicional na precisão do treinamento possa ser alcançada.

Performance do Classificador Base: Se um classificador fraco apresentar um erro igual ou superior a 0,5 (ou seja, não for melhor do que um palpite aleatório em classificação binária), o algoritmo para ou ignora esse estimador

Nota sobre *Overfitting*: Embora o AdaBoost possa ser forçado a parar ao atingir um erro zero, em conjuntos de dados com ruído, o uso de um número muito alto de iterações pode levar ao *overfitting*. Por isso, em práticas modernas, utiliza-se a parada antecipada (**early stopping**) baseada na validação de dados para evitar que o modelo se ajuste excessivamente aos ruídos.

Outros Algoritmos de Boosting

- **Gradient boosting:** funciona adicionando sequencialmente preditores a um conjunto, com cada um corrigindo os erros de seu antecessor. No entanto, em vez de alterar os pesos dos pontos de dados como o AdaBoost, o gradient boosting treina nos erros residuais do preditor anterior. O nome, gradient boosting, é utilizado pois combina o algoritmo de gradiente descendente e o método de boosting.

  Gradient boosting extremo, ou XGBoost: o XGBoost é uma implementação do gradient boosting projetada para velocidade e escala computacional. O XGBoost aproveita vários núcleos na CPU, possibilitando que o aprendizado ocorra em paralelo durante o treinamento.

Exemplo de codigo (Python)

```
from sklearn.ensemble import
HistGradientBoostingClassifier

gb = HistGradientBoostingClassifier(
    learning_rate=0.05,
    max_depth=6,
    max_leaf_nodes=31,
    min_samples_leaf=20,
    l2_regularization=0.0,
    early_stopping=True,
    random_state=42
)

gb.fit(X_train, y_train)
proba = gb.predict_proba(X_test)[:, 1]
```

Vantagens do Boosting

1. ***Desempenho aprimorado:*** *ele reduz efetivamente o viés e a variância, resultando em previsões mais precisas e robustas.*
2. ***Capacidade de lidar com dados complexos:*** *o Boosting pode lidar com padrões de dados complicados, incluindo correlações e interações não lineares, tornando-o apropriado para uma ampla gama de aplicativos de aprendizado de máquina, como classificação, regressão e classificação.*
3. ***Robustez ao ruído:*** *Quando comparado a outras técnicas de aprendizado de máquina, o Boosting é menos vulnerável ao ruído nos dados de treinamento, uma vez que se concentra em amostras mal classificadas e dá maior peso a elas, reduzindo efetivamente o impacto de amostras ruidosas nas previsões finais.*
4. ***Flexibilidade:*** *Os algoritmos de Boosting são versáteis e podem ser empregados com uma variedade de modelos básicos e funções de perda, permitindo a personalização e adaptação a vários domínios de problemas.*
5. ***Interpretabilidade:*** *Embora os modelos de Boosting sejam frequentemente referidos como modelos de "caixa preta", eles podem, no entanto, fornecer alguma interpretabilidade por meio de classificações de importância de recursos, o que pode ajudar na compreensão do valor relativo de vários recursos no processo de previsão.*

Algumas aplicações notáveis

- **Detecção de Rostos:** O AdaBoost tem sido utilizado com sucesso em sistemas de detecção de rostos. Ao combinar múltiplos classificadores fracos, o AdaBoost pode detectar rostos com precisão em imagens e vídeos.
- **Avaliação de Risco de Crédito:** O Gradient Boosting tem sido empregado na indústria financeira para avaliação de risco de crédito. Ao analisar diversas características dos solicitantes de empréstimos, como histórico de crédito e renda, os modelos podem prever a probabilidade de inadimplência e auxiliar nas decisões de crédito.
- **Diagnóstico Médico:** Algoritmos de Boosting têm sido aplicados no domínio médico para diagnóstico e prognóstico de doenças. Ao combinar múltiplos classificadores fracos baseados em dados dos pacientes, como sintomas e resultados de testes, os modelos de reforço podem auxiliar na detecção precoce e previsão de diversas doenças.
- **Detecção de Fraudes:** Algoritmos de aprimoramento têm sido usados em sistemas de detecção de fraudes para identificar atividades suspeitas. Ao analisar padrões e anomalias nos dados transacionais, os modelos de reforço podem sinalizar possíveis transações fraudulentas e ajudar a prevenir perdas financeiras.
- **Previsão de Abandono de Clientes:** Nos setores de comércio eletrônico e telecomunicações, algoritmos de aprimoramento têm sido empregados para prever a rotatividade de clientes. Ao analisar o comportamento dos clientes e os padrões de uso, os modelos de reforço podem identificar clientes que provavelmente vão descontinuar seus serviços, permitindo que as empresas tomem medidas proativas para retê-los.

Bibliografia desta Aula

Para aprofundamento nos assuntos desta aula, segue a seguinte referência bibliográfica

- Edirlei Soares de Lima - INF 1771 – Inteligência Artificial – Slides da Aula 15 – Support Vector Machines (SVM) – PUC-RIO
- Protopapas, P; Rader, K and Tanner, C. Slides of CS109A Introduction to Data Science - IACS - Harvard University
- Brijesh Soni - Understanding Boosting in Machine Learning: A Comprehensive Guide – Available at: https://medium.com/@brijesh_soni/understanding-boosting-in-machine-learning-a-comprehensive-guide-bdeaa1167a6
- Livro referência: Russel & Norvig – Artificial Intelligence – A modern Approach – 4th edition (2022) – Capitulo 19

- Alguns slides foram baseados em:
  - Aulas do Prof. Dr. Danilo H. Perico - FEI